The C programming

Constants, Variables and Keywords

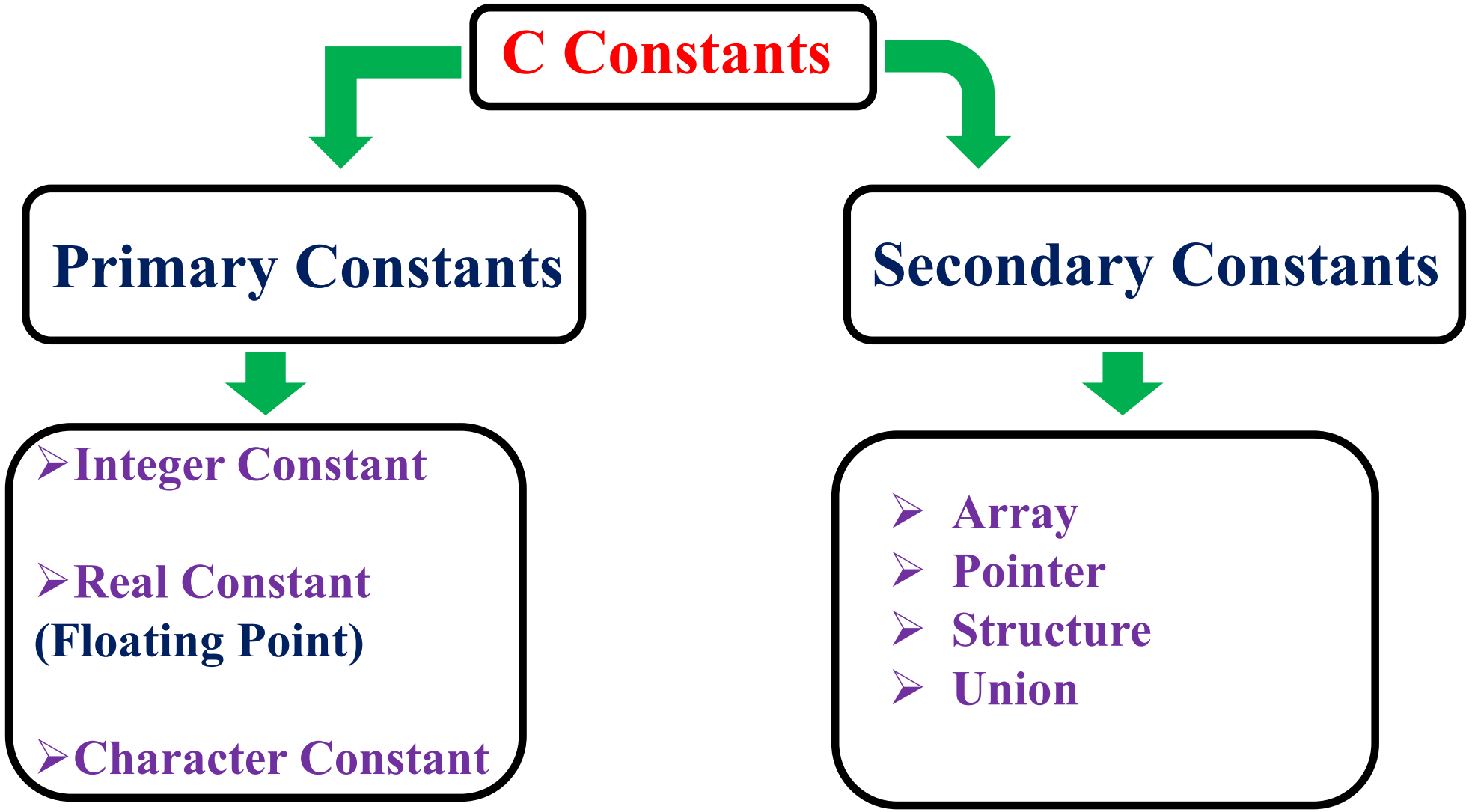
- **Constants: That remains always fixed**

- **Variable: That may change.**

- **Keywords: Words whose meaning has already explained to the complier, hence keywords can't be used as variable**

e.g. int , for, while, struct, void, main, volatile etc.

Classification of Constants



Integer, Real and Character Constant

Integer Constants

- It should have at least 1 digit
- Can be positive or negative , if no sign mentioned it is treated as positive one.
- Decimal point is not allowed.
- Commas, blanks are not allowed.
- E.g. 1022,-56,-5612, etc.

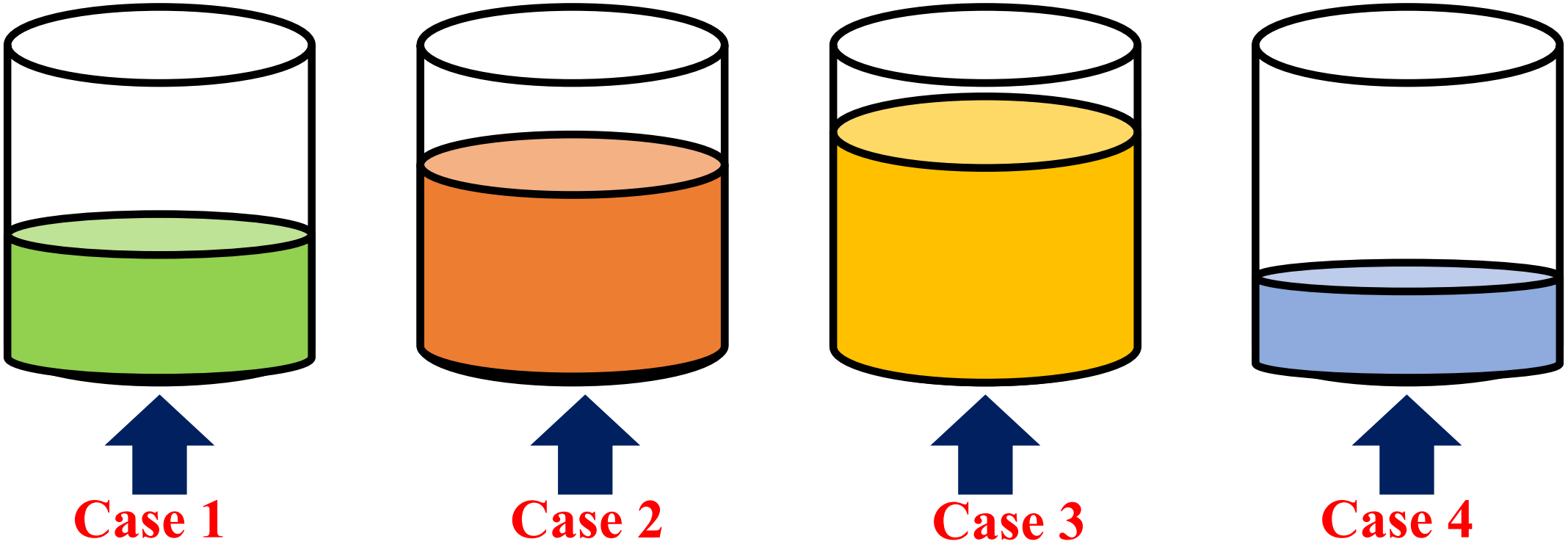
Real Constants

- It should have at least 1 digit
- Can be positive or negative , if no sign mentioned it is treated as positive one.
- Decimal point is allowed
- Commas, blanks are not allowed.
- E.g. +123.45,-56.45, etc.

Character Constants

- Single digit, single alphabet, single special symbols enclosed within single inverted commas.
- Both the inverted commas should be point to left ' '
- Max. length of character constant is 1 character
- E.g. 'A', '=', '8', etc.

Variables



Variables

Before using any variable it should be declared

```
int var;
```

```
float var1;
```

Where int , float: data types

var , var1: name of variables

Remember there is a semicolon after declaration because it makes the compiler understand the line separation.

Variable Declaration and Definition Section

```
int a, b, c;   {variable declaration}
a=10; {variable define}
b=20; {variable define}
c=45; {variable define}
```

Case 1

```
char x, y, z;   {variable declaration}
x='a'; {variable define}
y='b'; {variable define}
z= 'n'; {variable define}
```

Case 3

```
float x, y, z;   {variable declaration}
x=10.2; {variable define}
y=20.9; {variable define}
z=45.4; {variable define}
```

Case 2

```
int x=10; {variable dec. and define.}
int u=90; {variable dec. and define.}
```

Case 4

Rules associated to Variables

- **First character in the variable should be an alphabet or an underscore, Don't start with any of the digit**

e.g. iiitb: Allowed

5iiitb: Not Allowed

- **Var, var, VAR, VaR all are different variables**

- **Among Special symbols only underscore is allowed.**

e.g. iiit_b: Allowed

iiit@b: Not Allowed

- **Keywords names are not used in variable.**

e.g. switch, for, while, etc.

SWitch: Allowed

switch: Not Allowed

int my age

Fill in the valid name of a variable

int ----------

- **Switch**
- **695**
- **Int**
- **for**
- **@var**

Programming Section

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <math.h>
.............
```

Header Files according to task as input, output, maths, etc.

```
int main () {
}
float main () {
}
void main () {
}
```

- int main states it would return integer value
- float main states it would return float value
- void main states it would not return anything

Programming Examples

```
#include<stdio.h>
 int main(){
printf (“ IIIT BHOPAL”);
        return 0;
}
```

O/P: IIIT BHOPAL

```
#include<stdio.h>
 int main(){
printf (“ iiit bhopal”);
        return 0;
}
```

O/P: iiit bhopal

```
#include<stdio.h>
 int main(){
        int var;
       var = 3;
 printf (“ %d”, var);
        return 0;
}
```

O/P: 3

```
#include<stdio.h>
 int main(){
        int var = 3;
        var=2;
 printf (“ %d”, var);
        return 0;
}
```

O/P: 2

```
#include<stdio.h>
 int main(){
        int var = 3;
        int var2;
        var2=var+1;
 printf (“ %d”, var2);
        return 0;
}
```

O/P: 4

%d: value stored at int var

%f: value stored at float var

%c: for char type

printf and scanf commands

printf(): This command gives the output to the screen which is already mentioned in the program.

e.g. printf (" IIIT BHOPAL");
printf (" iiit bhopal");
printf (" %d", var2);
printf (" %f", var2);

scanf(): Using this command input is take from the user using keyboard.

e.g. scanf ("%d", &k);
scanf ("%f", &k);

&k : states the memory location of variable and inside that location value is stored.

Types of Instructions

Type Declaration Ins.

- Using this Instruction data type of variable is declaration.

- int a;
- float b;
- char c;

Arithmetic Ins.

- Using this Instruction variable name is defined on LHS of '=' and RHS its value is assigned.

- int a;
- a = 10;
- float b;
- b = 23.5;

Control Ins.

- This Instruction control the sequence of execution of different statement mentioned in the program

- if statement
- else statement ,etc.

Operators

Name of Operators	Operators
Arithmetic Operator	+, -, *, /, %
Increment / Decrement Operator	++, --
Relational Operator	== , !=, > , < , >= , <=
Logical Operator	&&, || , !
Bitwise Operator	>> , << , & , ^ , | , ~
Assignment	=, +=, -=, *=, /= ,etc.

Operators

- 10/2 = 5 { /: This gives the quotient}
- 10%2 = 0 {%: This gives the remainder}
- = : {This is assignment operator} (a = 5 , x = 120.4 , v = 'a')
- == : {This is equality operator}
- a+=5; { a = a+5}, b*=5; { b = b*5}
- != : {Not equal to}

Data Types

Name of Data Type	Size
Integer (int)	4 Bytes (32 bits)
Short integer Long integer	2 Bytes (16 bits) 8 Bytes (64 bits)
Float (float)	4 Bytes (32 bits)
Double	8 Bytes (64 bits)
Long Double	12 Bytes (96 bits)
Character (char)	1 Byte (8 bits)

1 Byte : 8 bits

Examples

```
#include <stdio.h>
 int main(){
     float v=3.151612;
     printf("%f\n",v);
     printf("%.1f\n",v);
     printf("%.3f\n",v);
     printf("%.5f\n",v);
     printf("%.2f\n",v);
}
```

O/P: 3.151612
3.1
3.151
3.15161
3.15

```
#include <stdio.h>
 int main(){
     int var = 4/9;
     printf("%d\n",var);

     float var2 = 4/9;
     printf("%.2f\n",var2);

     float var3 = 4.0/9.0;
     printf("%.2f\n",var2);
}
```

O/P: 0
0.00
0.44

Operator Precedence and Associativity

Operator	Associativity
*, /, %	Left to Right (High Preference)
+ , -	Left to Right (Low Preference)
=	As it is (Lowest Preference)

```
#include <stdio.h>
 int main(){
      int a=2, b=3, c=4, d=5;
      printf("%d\n", a*b/c);
      printf("%d\n", a+b-c);
      printf("%d\n", a+b*d-c%a);
}
```

Case 1: Associavity
follows a*b/c
(a*b)/c
6/4 : 1 O/P = 1

Case 2: Associavity
follows a+b-c
(a+b)-c
5-4 : 1 O/P = 1

Case 3: Associavity
follows a+b*d-c%a
a+(b*d)-(c%a)
2+15-0 : 17-0 O/P = 17